CV Best Practices, Augmentation Techniques

Sattam Altwaim
satttam.tuuaim@kaust.edu.sa



جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology



KAUST Academy & Mawhiba
AI Olympiad Training (Beginners)
Week 4, Day 27, Self-study, Olympiad practice

15 October 2026

Table of Contents

1. Where This Fits
2. Learning Outcomes
3. The Routine
4. Common Tricks
5. Common Mistakes
6. Tips
7. Common Practices
8. Olympiad Practice
9. Check Yourself
10. Limitations and What Comes Next
11. Summary
12. References

From the Lectures

- **This week:** convolutions, pooling and the CNN as a class (Day 22); blocks, stages, the GAP head and the augmentation catalogue with its label-safety rule (Day 23); YOLO, U-Net, IoU and mAP (Day 24); shapes, shape errors and the classification pipeline from folder to submission (Day 25); the metrics by hand and the settings that move them (Day 26).
- **This deck adds** the habits that decide a vision score on a clock, each taken from a published source: what the winners of a 2,245-competitor detection challenge had in common; what the YOLO documentation says a dataset and a training run should look like; which classification tricks are worth how many points, measured; where augmentation silently breaks labels, boxes and masks; how a real Olympiad task (IOAI 2025, *Chicken Counting*) was posed and solved; and the mistakes that cost the most.
- **How to use it:** one item per frame: the situation, then the move, with code when the move is code and the source under it. Every number was measured by someone and can be looked up; nothing here is the author's guess.

Why Go Deeper

What the evidence says decides a vision competition

- **Global Wheat Challenge 2020** (Kaggle, 2,245 competitors, 6,515 photos, 275,187 wheat heads): the winners used "standard existing open-source architectures" and all of them used pseudo-labelling and test-time augmentation with weighted boxes fusion; their gains came from the data side, not from new networks (David et al., 2023).
- **IOAI 2025, Chicken Counting:** 100 training photos, a pretrained feature extractor handed out, a metric of exp(−mean relative error), the committee's baseline at 0.71 and its best at 0.89. The whole task is the pipeline, the split and the loss; the architecture is given.
- **Image classification:** the same ResNet-50 goes from 77.2% to 79.3% top-1 with training tricks alone, no change to the network (He et al., 2019).

After this deck you can

run a vision task from a blank notebook to a defensible submission in the order that wins points, choose augmentation that keeps the labels true, and avoid the five mistakes that make a good model score badly.

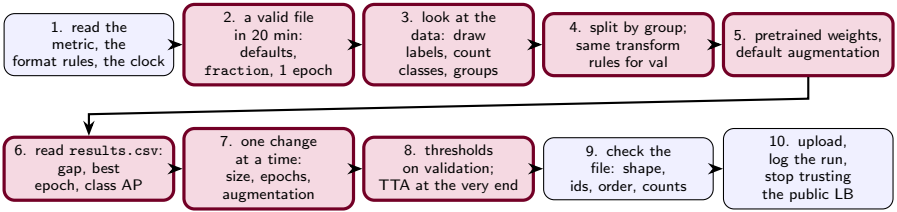
⇒ Learning Outcomes

By the end of this self-study deck, you will be able to:

- **Run** the vision routine on a clock: a valid file first, pretrained weights and defaults, then one change at a time judged on validation.
- **Choose** augmentation for the task from the YOLO defaults and the label-safety rule, and apply it to boxes and masks together with the image.
- **Quote** what each trick is worth, with its source: cosine schedule, label smoothing, mixup and CutMix, resolution, test-time augmentation, box fusion.
- **Split** data without leakage when images come in groups (patients, videos, near-duplicates), and validate on held-out images only.
- **Set** the thresholds and sizes that the metric rewards: confidence for mAP, IoU for touching objects, mask threshold for Dice, the same image size at train and test.
- **Check** a submission against the task's format rules before uploading, and read a task statement for its metric, its data and its hidden traps.

CV Best Practices: **The Routine**

The Vision Run Routine



- Steps 1, 9 and 10 are Day 13's and Day 20's routine; the purple steps are what changes with images: every epoch is minutes, not seconds, so the order of the moves is the budget. The YOLO documentation's own advice is the same: "train with default settings first", then change one thing (Ultralytics, tips for best training results).
- The rest of the deck is the purple steps, one habit per frame.

What the Judge Reads, by Task Type

task	metric you will meet	what moves it most	the deck that taught it
classification	accuracy, macro F1	the split and the transforms; label smoothing and schedule; class weights for F1	Days 23, 25; Day 09
detection	mAP@0.5, mAP@[.5:.95], or an accuracy over IoU thresholds (wheat challenge)	image size for small objects; the whole ranked list submitted; NMS IoU for touching objects	Days 24, 26
segmentation	Dice or IoU per class, mean IoU	the mask threshold; BCE + Dice loss; the group split	Days 24, 26
counting	$\exp\left(-\frac{1}{n}\sum \lvert y_i - \hat{y}_i\rvert / y_i\right)$ (IOAI 2025)	a density map whose sum is the count; no threshold; the relative error, so small counts matter most	today

- Read the formula before anything else. The counting metric divides by the true count: an error of 5 chickens on a photo of 10 costs ten times an error of 5 on a photo of 100. A negative density or a wrong array shape scores zero: "Any result that does not meet the specified size will be considered invalid" (IOAI 2025 statement).
- The wheat challenge scored an image-level accuracy averaged over IoU thresholds 0.5 to 0.75 in 2020 and at 0.5 only in 2021 "to favor detection" (David et al., 2023): the same boxes, a different ranking.

CV Best Practices: **Common Tricks**

Trick: Start From Published Weights, at the Defaults

When it applies

Every detection or segmentation task with hundreds or a few thousand images, which is every Olympiad task. The YOLO documentation: pretrained weights for "small to medium datasets", from scratch only for large ones. IOAI 2025 handed out a pretrained feature extractor with the Chicken Counting task.

What to do

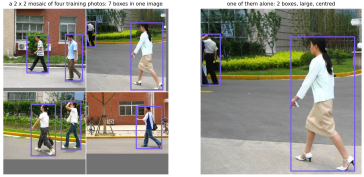
Load the COCO weights and train with the defaults before touching anything: "train with default settings first". The first run is the baseline every later change is judged against.

```
model = YOLO("yolov8n.pt")       # COCO
    weights
model.train(data="task.yaml", epochs=30,
            imgsz=640, batch=-1)   #
                defaults
# classification, Week 5's way:
backbone = torchvision.models.resnet18(
    weights="DEFAULT")
backbone.fc = nn.Linear(512, n_classes)
```

The wheat-challenge winners started from off-the-shelf models too: "standard existing open-source architectures, such as EfficientDet, Faster-RCNN, Yolo-v5, and Yolo-v3" (David et al., 2023). The classification version, a pretrained backbone with a new head, is Week 5's first lecture.

Sources: Ultralytics, *Tips for best training results*; IOAI 2025 Task 2 statement; David et al. (2023).

Trick: Augment With the Defaults, Then Move the Dials



Source: four Penn-Fudan photos tiled by `scripts/day27_practice_figures.py`, boxes shifted with their tiles; Mosaic as defined by Bochkovskiy et al. (2020).

Mosaic "mixes 4 training images": more objects, more scales, objects at the edges; on by default, off for the last 10 epochs.

argument	default, meaning
`hsv_h`, `hsv_s`, `hsv_v`	0.015, 0.7, 0.4: colour jitter
`degrees`	0: rotation
`translate`, `scale`	0.1, 0.5: shift, zoom 1 ± 0.5
`fliplr`, `flipud`	0.5, 0: flip probabilities
`mosaic`, `mixup`	1.0, 0: four tiled; two blended

```
model.train(data="task.yaml", epochs=40,
            degrees=10, flipud=0.5,   #
                aerial
            fliplr=0.0)               #
                 arrows
```

The documentation's rule: keep colour jitter; geometry when the camera angle varies; `flipud` only "when objects can appear upside down"; "start with a small set".

Sources: Ultralytics, *YOLO data augmentation guide* (defaults as listed); Bochkovskiy, Wang and Liao (2020), Section 3.3.

Trick: Classification Tricks With Measured Gains

ResNet-50 on ImageNet, top-1	%	source
baseline, 120 epochs	77.16	He et al. 2019, Table 6
+ cosine learning-rate decay	77.91	
+ label smoothing $\varepsilon = 0.1$	78.31	
+ mixup $\alpha = 0.2$, 200 epochs	79.15	
baseline, 300 epochs	76.3	Yun et al. 2019, Table 1
+ Cutout	77.1	
+ mixup	77.4	
+ CutMix	78.6	

Two independent papers, the same order of magnitude: a schedule and a loss trick are worth about one point each; mixing images is worth one to two, but the authors needed 200 to 300 epochs because "the mixed examples ask for a longer training progress to converge better".

What to do on a clock

Cosine decay and label smoothing cost nothing: two lines, no extra epochs. Mixup and CutMix only pay when you can afford the long schedule; on a three-hour task, try them last.

```
loss_fn = nn.CrossEntropyLoss(
    label_smoothing=0.1)
opt = torch.optim.AdamW(
    model.parameters(), lr=1e-3)
sched = CosineAnnealingLR(
    opt, T_max=EPOCHS)
for epoch in range(EPOCHS):
    train_one_epoch(...)
    sched.step()
```

Sources: He, Zhang, Zhang, Zhang, Xie and Li (2019), Tables 4 and 6; Yun et al. (2019), Tables 1 and 3; Zhang et al. (2018) for mixup; DeVries and Taylor (2017) for Cutout.

Trick: Give Small Objects Pixels

When it applies

Objects of 10 to 30 pixels: distant signs, wheat heads, cells. Resizing a 1280-pixel photo to 640 halves every object: a 20-pixel sign becomes 10, about one cell of the finest YOLO map (stride 8).

What to do

Train at a larger `imgsz`: the documentation keeps 640 for COCO but datasets "with many small objects benefit from `--img 1280` or higher"; then "run inference at the same size used for training". The wheat challenge worked at 1,024 pixels; one 2021 winner "upsampled the images to 1,600 px before prediction".

```
model.train(data="task.yaml", imgsz=1024,
            batch=-1, epochs=30)   # auto
                batch
m = model.val(imgsz=1024)          # same
      size
r = model.predict(path, imgsz=1024)
```

- **The price:** 1024 costs 2.6× the memory and time of 640; `batch=-1` picks the batch that fits. Time one epoch first.
- **Classification:** trained at 224 with random crops, a ResNet-50 scores 78.4% tested at 288 against 77.0% at 224 (Touvron et al., 2019). Try one size up at test, on validation first.

Sources: Ultralytics, *Tips for best training results*; David et al. (2023); Touvron, Vedaldi, Douze and Jégou (2019).

Trick: Test-Time Augmentation, Measured Before You Pay for It



Source: YOLOv8n's published COCO weights on Penn-Fudan photo `FudanPed00010`, `conf=0.25`, without and with `augment=True`; `scripts/day27_practice_figures.py`.

What it is: predict on the photo and on flipped, rescaled copies; merge the boxes before NMS, or average a classifier's softmax. Measured by Ultralytics, YOLOv5x on COCO: mAP@[.5:.95] 0.504 $\rightarrow$ 0.516 at 22.4 $\rightarrow$ 80.6 ms per image; all six wheat-challenge winners used it with box fusion. **Measure it yourself:** on the first 60 Penn-Fudan photos the plain pass already finds all 129 people and TTA only adds boxes (249 $\rightarrow$ 282). It pays where recall on small objects is the problem and costs 3.6$\times$ the time: last move of the day.

```
r = model.predict(path, augment=True)
# a classifier: average two views
p = (softmax(net(x))
     + softmax(net(x.flip(-1)))) / 2
```

Sources: Ultralytics, *Test-time augmentation* tutorial (YOLOv5x numbers); David et al. (2023); the Penn-Fudan count from the figure script.

Trick: Choose Every Threshold on Validation, for the Metric

When it applies

Every task has a number you set after training: a detector's confidence cut and NMS IoU, the mask threshold, the class decision of an unbalanced classifier. None is learned; each moves the metric (Day 26).

metric	setting	rule
mAP	`conf`	low (0.001 to 0.05): the whole list
mAP, touching objects	NMS `iou`	raise from 0.5 towards 0.7
Dice, IoU	mask threshold	sweep 0.3 to 0.7 on validation
macro F1	class weights	weight the loss, pick the epoch by F1
count by density	none	the sum of the map is the count

```
probs = []
with torch.no_grad():
    for x, y in val_loader:
        logits = model(x.to(device))
        p = torch.sigmoid(logits).cpu()
        probs.append((p, y))
best = (0, None)
for t in [0.3, 0.4, 0.5, 0.6, 0.7]:
    ds = [dice((p > t).float(), y)
          for p, y in probs]
    best = max(best, (np.mean(ds), t))
print(best)   # (validation Dice, threshold)
```

One forward pass over validation, cached; the sweep is free. The same loop serves a detector's `conf` and a two-class F1 threshold. The test predictions use exactly the number you picked.

Sources: Day 26 of this course; IOAI 2025 Task 2 reference solution (count = sum of the predicted density, no threshold).

Named, Not Taught: Pseudo-Labels and Box Fusion

Pseudo-labelling

Predict on the unlabelled test photos, keep the confident predictions as if they were labels, train again on both. “All the winning solutions used pseudo-labeling” in the wheat challenge (David et al., 2023). It needs a model that is already good, a confidence cut chosen on validation, and the time for a second training run.

Weighted boxes fusion

Instead of deleting overlapping boxes (NMS), average their coordinates weighted by confidence, over several models or TTA views. An ensemble of detectors on COCO: best single model 52.1, fused 56.4 mAP; on a single model’s TTA views the gain was small, $0.5233 \rightarrow 0.5262$ (Solovyev et al., 2021).

Why they are only named

Both are second-day moves: they multiply the training time and only help a model that already works. On a three-hour clock, every frame before this one comes first. Know the words so that a write-up or a teammate’s suggestion is not a mystery.

Sources: David et al. (2023); Solovyev, Wang and Gabruseva (2021).

CV Best Practices: **Common Mistakes**

Mistake: Leakage Through the Split

The case

Day 24's MRI slices are the data of Cheng et al. (2015): "We collected 3064 slices from 233 patients." Neighbouring slices of one patient look alike; a random split by slice puts a patient on both sides and the validation Dice measures memory. The paper split by **patient**: "slices from the same patient will not exist in the training and testing set simultaneously."

- The same trap: frames of one video, several photos of one plant or scene, near-duplicate downloads. The hidden test set is usually a new group, so only a grouped split predicts it.

```
from sklearn.model_selection import
    GroupShuffleSplit
gss = GroupShuffleSplit(1, test_size=0.2,
                        random_state=0)
tr, va = next(gss.split(files, groups=
    patient))
assert not set(patient[tr]) & set(patient
    [va])
```

No group id given? Look for one: file-name prefixes, folder names, capture dates. If two images look like neighbours, treat them as one group.

Source: Cheng et al. (2015), PLoS ONE 10(10); the Kaggle copy of the same data is Day 24's Lab 2 set.

Mistake: Transforming the Image but Not Its Labels



Source: Penn-Fudan photo `FudanPed00005`; `torchvision.transforms.v2` on the image, its two boxes and its mask together, then the same image with the untransformed labels; `scripts/day27_practice_figures.py`.

```
from torchvision import tv_tensors
from torchvision.transforms import v2
boxes = tv_tensors.BoundingBoxes(b, format="
    XYXY",
                                 canvas_size
                                     =(H, W)
                                     )
mask = tv_tensors.Mask(m)
img, boxes, mask = tf(img, boxes, mask)  #
    together
```

A flip moves every box and mask pixel; a crop drops boxes outside it. `transforms.v2` does this once labels are wrapped as `tv_tensors`; Albumentations through `BboxParams`, whose documentation warns that a wrong format gives wrong boxes with no error, "so visualize a few augmented samples before training". YOLO's loader does it all for you.

Sources: torchvision `transforms.v2` documentation; Albumentations, *Bounding boxes augmentation*.

Mistake: Augmentation That Changes the Answer

The case

A horizontal flip turns "keep left" into "keep right" and a 6 into a mirrored 6; a vertical flip puts the sky under a street; a crop can cut the only object out of a photo, or change the count of a counting photo. Mosaic pastes partial objects at tile borders: use it "only if partially occluded objects or multiple objects per image are acceptable and don't change the label".

- YOLO's defaults are the safe side: `flipud=0`, `degrees=0`, `fliplr=0.5`. Turn `fliplr` off for signs, text and left/right classes; `flipud` on for aerial or microscope images. Day 23's test: is the augmented picture, with this label, a fair test image?

Sources: Ultralytics, *YOLO data augmentation guide*; Day 23's label-safety table.

```
# arrows, digits, text
model.train(data="signs.yaml", fliplr
    =0.0)
# satellite tiles, cells: no up or down
model.train(data="tiles.yaml", flipud
    =0.5,
            degrees=180)
# counting: crop image AND density map
img, dens = tf(tv_tensors.Image(img),
               tv_tensors.Mask(dens))
```

For a density map a flip keeps the sum; a crop changes it by what it removes, which is right if the map was cropped too; a resize changes the sum unless the values are rescaled. Check `dens.sum()` before and after.

Mistake: One Size for Training, Another for the Test

The case

A detector trained at `imgsz=640` and run at 320 sees every object at half size; the small ones vanish and mAP drops with no error message. Day 25's classifier trained on 96-pixel photos and fed 150-pixel ones stopped with a shape error; a GAP head would have run and scored badly instead.

- The documentation's rule: "run inference at the same size used for training". Put the size in one variable and use it in `train`, `val` and `predict`.
- The one deliberate exception is a controlled step up at test time (Touvron et al., 2019), measured on validation, never assumed.

Sources: Ultralytics, *Tips for best training results*; Touvron et al. (2019); Day 25.

```
SIZE = 640
model.train(data="task.yaml", imgsz=SIZE)
model.val(imgsz=SIZE)
model.predict(test_dir, imgsz=SIZE)
# classification: one Resize for val and
    test
tf_eval = transforms.Compose([
    transforms.Resize((SIZE, SIZE)),
    transforms.ToTensor(),
    transforms.Normalize(MEAN, STD)])
```

The same goes for `Normalize`: the test transform is the validation transform, to the number.

Mistake: Validating on the Training Images

The case

The IOAI 2025 Chicken Counting reference notebook builds its validation loader from the training set: `val_loader = DataLoader(train_dataset, ...)`. With 100 labelled photos and hidden test labels that is tempting, and it means the saved "best" checkpoint is the one that fits the training photos best; it says nothing about the hidden 200.

- Hold out photos even when there are few: 20 of 100 is a noisy but honest number, 0 of 100 is no number. Choose epochs and thresholds on them; retrain on all 100 only as the last step, with the epoch count they chose.

Source: IOAI 2025, Task 2 reference solution notebook (`Chicken_Counting_Solution.ipynb`).

```
idx = torch.randperm(100, generator=g)
train_ds = Subset(full, idx[20:])
val_ds = Subset(full, idx[:20])
# epochs and thresholds chosen on val_ds;
# last step: retrain on all 100
final = train(Subset(full, idx), epochs=
    best_epoch)
```

The metric is relative error: check the held-out photos with the fewest chickens separately, they dominate the score.

Mistake: A Good Model, a Worthless File

what happened	what the judge sees	the one-line check
boxes submitted above `conf=0.5` for a mAP task	the curve stops at low recall; AP loses the whole tail (Day 26)	`conf=0.001` at `val`, ≤ 0.05 at `predict`
test loader shuffled, or rows not in the sample's order	right predictions on the wrong rows	`(sub.image == sample.image).all()`
class index typed by hand	a permutation of the labels	`i2c` built from `class_to_idx`, printed
array of the wrong shape or negative densities	"considered invalid", score 0 (IOAI 2025)	`pred.shape == (100, 1, 180, 320)`, `pred.min() >= 0`
train transform left on at test	a random crop of every test photo	`print(test_ds.transform)`
last epoch instead of `best.pt`	an overfitted model	load `best.pt`, say so in the log

- Every row is a model that scored well on validation and badly on the leaderboard. Put the checks in one cell that runs before `to_csv` or `np.savez`, every time. The IOAI statement spells its format out because contestants lose it: read the format paragraph twice.

Sources: IOAI 2025 Task 2 statement; Days 25 and 26.

CV Best Practices: **Tips**

Tip: A Valid File in the First Twenty Minutes

Why

The Chicken Counting baseline, a pretrained extractor plus one convolution trained for 20 epochs, scored 0.71 against the committee's best of 0.89: a working pipeline is already most of the score, and a crash at upload time is all of it. Day 13's rule for tables holds twice for images, where one full run can take the whole budget.

What to do

Run the pipeline end to end on a slice of the data: `fraction=0.1`, one epoch, `imgsz=320`; write the file; check it with the format rules; upload it. Then scale up.

```
# minute 0-20: does everything connect?
model = YOLO("yolov8n.pt")
model.train(data="task.yaml", epochs=1,
            imgsz=320, fraction=0.1)
write_submission(model, test_dir)   # and
    check it
# minute 20+: the real run
model.train(data="task.yaml", epochs=40,
            imgsz=640, patience=10)
```

`fraction` trains on a subset "as a ratio or an image count"; `patience` stops after that many epochs without improvement, so a long run cannot eat the clock unnoticed.

Sources: IOAI 2025 Task 2 statement (scores); Ultralytics, *Train* mode arguments.

Tip: Look at a Training Batch Before the First Epoch

Why

Every mistake of the last section shows in one picture of a batch with its labels drawn on: a box beside its object, a mask off by a flip, an arrow pointing the wrong way. YOLO writes `train_batch0.jpg` into the run folder for this reason; for your own Dataset you draw it.

What to do

Eight augmented samples from `train_ds`, labels drawn, before training. Again after every change to the transform. Compare with eight from `val_ds`: those must look like plain photos.

```
fig, ax = plt.subplots(2, 4, figsize=(14,
    7))
for a in ax.flat:
    img, boxes, mask = train_ds[random.
        randrange(len(train_ds))]
    show(a, img, boxes, mask)      # Day
        24's helper
plt.show()
# YOLO: open runs/detect/train/
    train_batch0.jpg
```

Albumentations says it for boxes and it holds for everything: a wrong format gives no error and a quietly wrong dataset. Looking is the only test.

Sources: Ultralytics training outputs (`plots=True`); Albumentations, *Bounding boxes augmentation*.

Tip: The Settings That Buy Time

setting	default	what it buys
`batch=-1`	16	the largest batch that fits about 60% of GPU memory, chosen for you
`cache=True`	False	images decoded once into RAM; the disk stops being the bottleneck (Day 19's lesson)
`patience=10`	100	stops a run that stopped improving; the default would run to the end
`close_mosaic=10`	10	the last epochs on real photos, not mosaics; keep it
`cos_lr=True`	False	cosine decay, the free point of the classification table
`amp=True`	True	half-precision arithmetic on supported GPUs, about twice the speed; already on
`workers`	8	loader processes; Colab has two CPUs, so 2
`epochs`	100	the documentation's full recipe starts at 300; a clock allows 20 to 50, so make them count

- Time one epoch at the real `imgsz` and multiply before choosing `epochs`: the number must fit the clock with half an hour to spare for validation, thresholds, the file and one retry.
- The classification loop has the same levers: `num_workers`, a cached Dataset, `torch.autocast` for mixed precision, early stopping on the metric (Day 20).

Source: Ultralytics, *Train* mode arguments and *Tips for best training results*.

Tip: Log Every Run, Change One Thing

run	size	ep	aug	best ep	val	LB
1	320	1	default	1	0.41	0.39
2	640	30	default	24	0.78	0.75
3	640	30	fliplr 0	27	0.80	0.77
4	1024	30	fliplr 0	29	0.83	0.80
5	1024	30	+ TTA	29	0.84	0.80

The shape of a log, not a measurement: the columns are what matters.

Day 20's table with the vision columns: image size, epochs, the augmentation flags, the best epoch, the validation metric and the public score. Written by hand in a markdown cell, five seconds per run.

What to do

One change per row. A row that moves validation but not the leaderboard is noise or leakage; a row that moves the leaderboard but not validation says your split does not look like the test set (go back to the group split). Keep the `results.csv` folders; their names go in the log.

```
model.train(..., project="runs_task",
            name="r4_1024_noflip",
                 exist_ok=True)
```

The wheat-challenge analysis warns that on small hidden sets "luck may have also played some role in the final performance": a difference of a few thousandths is not a result.

Sources: Day 20; David et al. (2023).

CV Best Practices: **Common Practices**

Practice: The Recipe Card, by Task

	data and split	training	after training	source
classification	RandomResizedCrop, flip (if safe), ToTensor, Normalize; stratified split; eval transform for val	pretrained backbone (Week 5) or Day 23's recipe; AdamW 10^{-3}, cosine, label smoothing 0.1, 10 to 20 epochs; macro F1 or accuracy picks the epoch	TTA flip average; class threshold for F1	He et al. 2019; Days 23, 25
detection	YOLO label files and YAML; `fliplr` off for arrows; group split	`yolov8n/s.pt`, default augmentation, `imgsz` by object size, `patience`, `close_mosaic=10`	`conf` low for mAP, NMS `iou` by overlap, TTA last	Ultralytics docs; David et al. 2023
segmentation	image and mask through the same transform; patient or group split	U-Net or a YOLO-seg; BCE with `pos_weight` + Dice loss; Dice picks the epoch	threshold swept on validation; nearest resize back	Days 24, 26; Cheng et al. 2015
counting	density maps augmented with the image; relative-error metric	pretrained extractor + decoder; L1 loss on the map; held-out photos	count = sum; non-negative output; shape check	IOAI 2025 Task 2

Four cards, one routine: pretrained start, defaults, a group split, the metric on validation, one change at a time, thresholds last.

Practice: The Dataset Rules From the YOLO Documentation

What a good detection dataset looks like

- "$\geq$ 1500 images per class recommended"; "$\geq$ 10000 instances (labeled objects) per class recommended".
- Every instance labelled ("partial labeling" fails); boxes tight around the object.
- "about 0-10% background images to help reduce FPs": photos with no objects and no labels.
- Validation and test out of training; class distributions similar across splits; variety of light, season, angle.

When the task gives you 100 photos

You will not reach those numbers, which is why to read them: with 877 road-sign photos (Day 24) or 100 chicken photos (IOAI 2025) pretrained weights, augmentation, early stopping and a correct split decide everything; an architecture search is wasted time.

- Count instances per class on day one (Day 24's Lab 1: 783 speed limits, 91 stop signs); the class with 91 is where the noisy AP will be.
- Add background photos if the test set has empty frames: a detector never shown one invents objects in it.

Source: Ultralytics, *Tips for best training results* (dataset section), quoted.

Practice: What Wins Points, What Loses Them

Wins points	Loses points
a valid file from a one-epoch run in the first twenty minutes	the first upload in the last ten minutes, rejected for its shape
published weights and the default settings as the baseline	a network designed from scratch on 100 photos
a split by patient, video or scene	a split by slice that reports a Dice the test set never shows
eight augmented samples drawn and read before training	a flip that turns every left arrow into a right arrow, unnoticed
boxes and masks transformed with the image	labels that stayed where the image was
one `SIZE` for train, val and predict	640 for training, 320 for the test
the whole ranked list submitted for mAP	boxes above 0.5 confidence, the curve cut in half
thresholds swept on validation, TTA measured last	TTA switched on because a write-up said so
a run log with one change per row	five changes at once and a 0.004 “gain” adopted

CV Best Practices: **Olympiad Practice**

Practice Task: Counting From 100 Photos (IOAI 2025, Task 2)

Task, as posed

"A dataset with 100 images and their corresponding density maps": 720×1280 photos of chickens, each with a 180×320 density map whose sum is the count; two hidden sets of 100. Predict a non-negative map per photo; score $= \exp\big(-\frac{1}{n}\sum_i |y_i - \hat{y}_i|/y_i\big)$ over the totals. A pretrained four-layer extractor is provided ("a UNet can easily exhaust GPU memory"). Committee baseline 0.71, best 0.89.

- **Question 1:** which Week 4 task is this closest to, what does the network output, which of our losses and metrics apply, and why is there no threshold?
- **Question 2:** plan three hours with the routine: first twenty minutes, split, baseline, first change, what you would never try.
- **Question 3:** which augmentations keep the density map true, which break it, and what line checks it?
- **Question 4:** the metric divides by the true count: what does that mean for photos with few chickens?

Source: IOAI 2025 Individual Contest, Task 2 statement and reference solution, https://github.com/IOAI-official/IOAI-2025; quoted sentences are the statement's.

Solution Idea

1. Segmentation's shape with regression's loss: a map in, a map out, one channel, every pixel a non-negative number (a ReLU at the end, as the reference does). L1 between the maps; the count is the **sum** of the map, so no threshold: the metric reads totals, not pixels. Dice and IoU do not apply; relative error on the count does.
2. **0 to 20 min:** load, draw three photos with their maps, confirm `dens.sum()` matches an eye count; the provided extractor plus one convolution, one epoch, write `submission.npz`, check shape $(100, 1, 180, 320)$ and non-negativity, upload. **Split:** 80/20 by photo (group by barn or camera if the files say so). **Baseline:** the statement's recipe, 20 to 30 epochs, Adam 10^{-4}, L1, best epoch by held-out relative error. **First change:** a small U-Net decoder (mind the memory warning) or the input size, one at a time. **Never:** a backbone from scratch on 100 photos; validation on the training photos.
3. Flips keep the sum if the map flips too; colour jitter touches only the photo; a crop is right only if the map is cropped identically; a resize of the map must be followed by rescaling its values. Check `abs(dens.sum() - dens_aug.sum()) < 1e-3` for every transform but crops.
4. An error of 5 on a true count of 10 costs 0.5; on 100 it costs 0.05. Sort the held-out photos by true count and print the relative error of the five smallest; if they are bad, weight the loss by $1/y_i$ to match the metric.

CV Best Practices: **Check Yourself**

Check Yourself

Answer before turning the page.

1. **Sources.** A teammate says "mixup always adds two points". What did the two papers measure, under what schedule, and what would you try first on a three-hour clock?
2. **Split.** You are given 2,000 photos of 50 shops, 40 photos each, and must classify the shop type. How do you split, and what goes wrong with a random split?
3. **Augmentation.** For each task name one augmentation that is safe and one that is not: digits on licence plates; satellite roof segmentation; counting people from a density map; detecting arrows on road signs.
4. **The file.** Your validation mAP@0.5 is 0.81; the leaderboard says 0.46. List three checks in the order you would run them.

Answers

1. He et al. (2019): mixup +0.84 top-1 on ResNet-50, but with 200 epochs instead of 120; Yun et al. (2019): mixup +1.1 and CutMix +2.3 at 300 epochs. Both are long-schedule results. First on a clock: cosine decay (+0.75) and label smoothing (+0.4), which cost no epochs; mixing images last, if time remains.
2. Group by shop: `GroupShuffleSplit` with the shop id as the group, 40 shops to train and 10 to validate. A random split by photo puts every shop on both sides; the network recognises the shop, not the type, and the validation accuracy is far above what new shops will give (the Cheng et al. patient rule).
3. Plates: brightness jitter safe, horizontal flip not (digits mirror). Roofs: flips and 180° rotations safe if the mask turns too, colour jitter that changes roof material colours risky. Counting: flips safe with the map, resize without rescaling the map's values not. Arrows: colour and small rotation safe, `fliplr` not.
4. (1) The confidence cut: was the file written with `conf` around 0.5? Rewrite with 0.01 and keep the whole list. (2) The size: did `predict` use the training `imgsz`? (3) The format and order: ids and class names against the sample file, boxes in pixels of the original photos, not of the resized ones. If all three pass, suspect the split: a leaky validation set.

CV Best Practices: **Limitations & What Comes Next**

Limitations

- **The measured gains are someone else's.** ImageNet with ResNet-50 and hundreds of epochs; COCO with YOLOv5x; 6,515 wheat photos. The direction carries over to a Kaggle set of 2,000 photos, the size of the gain does not: measure each move on your split.
- **The YOLO dataset numbers are targets, not thresholds.** Olympiad tasks hand out a hundred to a few thousand images; the rules tell you which regime you are in, not that the task is impossible.
- **Two winners' tools are only named.** Pseudo-labelling and weighted boxes fusion are documented here so you recognise them; teaching them would take a day each, and both come after everything else on a clock.

The common mistake

Adopting a trick because a write-up used it. Winners' lists are long because winners had days; the routine, the split and the file are where the points are on a clock. Every trick on these frames comes with the measurement that justified it; ask for the same from your own run.

What Comes Next

- **Friday:** the Week 4 competition.
- **Week 5:** why "start from published weights" works and how far to take it: transfer learning, layer freezing, fine-tuning, linear probes, and embeddings from CLIP; the classification card's first line, explained.
- **Week 6:** the same routine on text.

CV Best Practices: **Summary**

Summary

- **The routine:** metric and format first; a valid file in twenty minutes; look at the data; split by group; pretrained weights at the defaults; read the log; one change at a time; thresholds on validation; TTA last; check the file.
- **Tricks, with their price:** cosine and label smoothing are free points; mixup and CutMix need long schedules; a larger `imgsz` buys small objects at the square of the cost; TTA gives tenths of a point for 3.6× the time; pseudo-labels and box fusion are the winners' second day.
- **Augmentation:** YOLO's defaults are the safe start; move the dials to the data (`fliplr` off for arrows, `flipud` on for aerial); labels, boxes, masks and density maps travel with the image or the dataset is wrong; a drawn batch is the only test.
- **Mistakes:** the leaky split (3,064 slices, 233 patients); validating on training photos; a size for training and another for the test; the confident-only submission; the file that fails the format rule.
- **Sources, not habits:** every number here has a paper, a documentation page or a task statement behind it, and your own validation split is the one that counts.

CV Best Practices: **References & Resources**

References & Resources

[1] David, E. et al. (2023). Global Wheat Head Detection challenges: winning models and application for head counting problems. *Plant Phenomics* 5, 0059. Dataset of 6,515 images and 275,187 heads from 12 countries; 2,245 competitors in 2020; the winners' shared tools: pseudo-labelling, test-time augmentation with weighted boxes fusion, mosaic and mixup, standard architectures; the two metrics. https://pmc.ncbi.nlm.nih.gov/articles/PMC10795497/

[2] IOAI 2025, Individual Contest, Task 2 *Chicken Counting*: statement (`Chicken_Counting.ipynb`) and reference solution (`Chicken_Counting_Solution.ipynb`). 100 training photos with density maps, the exp(−mean relative error) score, the format rule, the provided feature extractor, the baseline 0.71 and best 0.89.
https://github.com/IOAI-official/IOAI-2025

[3] He, T., Zhang, Z., Zhang, H., Zhang, Z., Xie, J. and Li, M. (2019). Bag of tricks for image classification with convolutional neural networks. *CVPR*. Tables 4 and 6: cosine decay, label smoothing $\varepsilon = 0.1$, mixup $\alpha = 0.2$ with 200 epochs; 77.16 to 79.29 top-1.

[4] Yun, S., Han, D., Oh, S. J., Chun, S., Choe, J. and Yoo, Y. (2019). CutMix: regularization strategy to train strong classifiers with localizable features. *ICCV*. ResNet-50 at 300 epochs: baseline 76.3, Cutout 77.1, mixup 77.4, CutMix 78.6.

[5] Zhang, H., Cisse, M., Dauphin, Y. and Lopez-Paz, D. (2018). mixup: beyond empirical risk minimization. *ICLR*. DeVries, T. and Taylor, G. (2017). Improved regularization of convolutional neural networks with Cutout. arXiv:1708.04552.

[6] Bochkovskiy, A., Wang, C.-Y. and Liao, H.-Y. M. (2020). YOLOv4: optimal speed and accuracy of object detection. arXiv:2004.10934. Section 3.3: Mosaic "mixes 4 training images"; the bag-of-freebies ablations.

References & Resources (cont.)

[7] Touvron, H., Vedaldi, A., Douze, M. and Jégou, H. (2019). Fixing the train-test resolution discrepancy. *NeurIPS*. ResNet-50 trained at 224: 77.0% tested at 224, 78.4% at 288.

[8] Solovyev, R., Wang, W. and Gabruseva, T. (2021). Weighted boxes fusion: ensembling boxes from different object detection models. *Image and Vision Computing* 107. COCO ensemble 52.1 to 56.4 mAP; TTA of one model 0.5233 to 0.5262.

[9] Cheng, J. et al. (2015). Enhanced performance of brain tumor classification via tumor region augmentation and partition. *PLoS ONE* 10(10), e0140381. 3,064 slices from 233 patients; patient-level five-fold split. The Kaggle copy is Day 24's Lab 2 dataset.

[10] Ultralytics documentation: *Tips for best training results* (dataset rules, pretrained weights, defaults first, image size, `--patience`, `--cos-lr`); *YOLO data augmentation guide* (the hyperparameters and their defaults, when to use flips and mosaic); *Test-time augmentation* (YOLOv5x: 0.504 to 0.516 mAP, 22.4 to 80.6 ms); *Train* mode arguments (`batch=-1`, `cache`, `fraction`, `patience`, `close_mosaic`, `amp`). https://docs.ultralytics.com

[11] Albumentations documentation, *Bounding boxes augmentation*: `BboxParams`, coordinate formats, `min_visibility`, and the warning that a wrong format gives wrong boxes without an error. https://albumentations.ai/docs/

[12] torchvision documentation, `transforms.v2` and `tv_tensors`: transforms applied to images, bounding boxes and masks together. https://pytorch.org/vision/stable/transforms.html

References & Resources (cont.)

[13] Shorten, C. and Khoshgoftaar, T. M. (2019). A survey on image data augmentation for deep learning. *Journal of Big Data* 6, 60. The catalogue behind Day 23 and this deck.

[14] Wang, L., Shi, J., Song, G. and Shen, I. (2007). Object detection combining recognition and segmentation. *ACCV*. The Penn-Fudan photos of the three figures. https://www.cis.upenn.edu/~jshi/ped_html/

[15] Figures without another source: `slides-builder/scripts/day27_practice_figures.py` (joint augmentation of boxes and masks with `transforms.v2`, the four-photo mosaic, YOLOv8n's published weights with and without `augment=True` on Penn-Fudan photos, and the 60-photo count). No model was trained for this deck; the run log on the tips frame is a template, labelled as such.